เอกสารรายละเอียดโครงงาน Senior Project: AI-Driven Mock Interview System for OOP Learning Assessment

บทนำ

โครงงานนี้มุ่งพัฒนาระบบสัมภาษณ์จำลองที่ขับเคลื่อนด้วย AI สำหรับประเมินการเรียนรู้ Object-Oriented Programming (OOP) โดยใช้ Multi-Agent LLM System ระบบจะช่วยอาจารย์ในการประเมินความเข้าใจของนักศึกษาผ่านการสนทนาแบบโต้ตอบ พร้อมให้ Feedback ที่มีคุณภาและสอดคล้องกับ Course Learning Outcomes (CLO)

---

1. SYSTEM SPECIFICATION (ข้อกำหนดระบบ)

1.1 Functional Requirements

สำหรับอาจารย์ (Instructor Role):

1. **CLO Management**
   - กำหนดและแก้ไข Course Learning Outcomes สำหรับหลักสูตร OOP
   - แมป CLO กับ Bloom's Taxonomy (จำ, เข้าใจ, ประยุกต์, วิเคราะห์, ประเมิน, สร้างสรรค์)
   - ตัวอย่าง CLO: "นักศึกษาสามารถออกแบบและนำ class hierarchy ไปใช้โดยใช้หลักการ inheritance และ polymorphism"
2. **Question Bank Management**
   - **ป้อนโจทย์โดยตรง**: สร้างคำถามด้วยตนเอง ระบุ CLO ที่เกี่ยวข้อง ประเภทคำถาม และ rubric
   - **Import เนื้อหาการสอน**: อัปโหลด slides, บทความ, code examples (PDF, DOCX, Python)
   - **AI-Generated Questions**: ระบบใช้ RAG สร้างคำถามจากเนื้อหาที่นำเข้า
3. **Rubric Designer**
   - สร้าง rubric แบบกำหนดเอง (4-5 ระดับ)
   - ใช้ AI Agent ช่วยแนะนำ rubric ตามคำถามและ CLO
4. **Dashboard และ Analytics**
   - ดูผลการประเมินของนักศึกษา
   - วิเคราะห์ CLO achievement rate
   - ระบุ misconceptions ที่พบบ่อย

สำหรับนักศึกษา (Student Role):

1. **Interview Session**
    - ตอบคำถามผ่าน text หรือ code editor
    - รับคำถามที่ปรับตามระดับความเข้าใจ (adaptive)
2. **Feedback และ Results**
    - ดูคะแนนและ feedback ทันทีหลังสัมภาษณ์
    - Feedback ละเอียด: จุดเด่น, จุดปรับปรุง, แนะนำการศึกษา
3. **Practice Mode**
    - ฝึกซ้อมโดยไม่นับคะแนน

1.2 Non-Functional Requirements

Performance:

- Response time: คำถามถัดไป < 5 วินาที
- รองรับ 50 concurrent users
- Uptime: 99.5%

Usability:

- System Usability Scale (SUS) ≥ 72 Wikipedia Usability Geek
- รองรับภาษาไทยและอังกฤษ

Security:

- Authentication: SSO (Google Workspace)
- Role-Based Access Control
- FERPA compliant

Cost Efficiency:

- ต้นทุนต่อการประเมิน < $0.20
- รองรับ 1,000 assessments/เดือน

---

2. SYSTEM ARCHITECTURE

2.1 Multi-Agent Architecture

ระบบใช้ **Orchestrator-Worker Pattern** โดยมี 6 agents: Anthropic +4

1. Orchestrator Agent

- LLM: Claude 3.5 Sonnet
- รับคำสั่งจากอาจารย์และแบ่งงานให้ sub-agents
- จัดการ error handling

2. Question Generator Agent

- LLM: GPT-4o Mini
- สร้างคำถาม OOP ที่สอดคล้องกับ CLO
- ใช้ RAG ดึงข้อมูลจากเนื้อหาที่นำเข้า

3. Rubric Designer Agent

- LLM: Claude 3.5 Sonnet
- สร้าง rubric สำหรับแต่ละคำถาม
- กำหนด criteria และ point distribution

4. Interviewer Agent

- LLM: GPT-4o
- นำเสนอคำถามและถามคำถามติดตาม
- ใช้ Socratic Method

5. Evaluator Agent

- LLM: Claude 3.5 Sonnet
- ประเมินคำตอบตาม rubric
- สร้าง feedback เชิงลึก

6. Code Executor Agent

- LLM: GPT-4o Mini
- รัน code ใน sandbox
- ทดสอบด้วย test cases
- วิเคราะห์ code quality

2.2 Framework: LangGraph

เหตุผลที่เลือก:

- Graph-based workflow เหมาะกับ interview flow ที่มี branching (Medium +3)
- State management ดี (Composio) (GitHub)
- Production-ready พร้อม monitoring (Turing)

2.3 RAG System

Components:

- Document Processor: แยก text และ code
- Embedding Model: OpenAI text-embedding-3
- Vector Database: Weaviate
- Retriever: Hybrid search (semantic + keyword)

2.4 Database Design

PostgreSQL Schema:

- users, courses, clos, questions, rubrics
- interview_sessions, student_attempts, responses
- clo_achievements

---

3. REQUIRED KNOWLEDGE

3.1 Large Language Models

- Transformer architecture
- Tokenization, embeddings
- Temperature, sampling
- Context window limitations

แนะนำ: Coursera "Generative AI with LLMs"

3.2 Multi-Agent Systems

- Agent coordination patterns Microsoft +3
- LangGraph, AutoGen, CrewAI comparison Medium +2
- Message passing, shared state

แนะนำ: LangGraph Official Tutorial

3.3 Prompt Engineering

- Few-shot learning Coursera DataCamp
- Chain-of-thought DataCamp
- Role prompting AVID Open Access
- Structured output

แนะนำ: DeepLearning.AI "Prompt Engineering"

3.4 Educational Assessment

- Bloom's Taxonomy
- Formative vs Summative Skillshub
- Constructive Alignment
- Feedback principles (Hattie & Timperley)

แนะนำ: "Understanding by Design"

3.5 OOP Concepts

- Encapsulation, Inheritance, Polymorphism, Abstraction
- SOLID Principles
- Design Patterns
- Common misconceptions

แนะนำ: "Head First Design Patterns"

3.6 RAG

- Vector embeddings
- Vector databases
- Retrieval strategies
- Chunking strategies

แนะนำ: Coursera "RAG" by DeepLearning.AI Coursera DeepLearning.AI

---

4. TECHNOLOGY STACK

4.1 LLM APIs

Primary: Claude 3.5 Sonnet

- $3/M input, $15/M output Vantage
- Use: Evaluator, Rubric Designer

Secondary: GPT-4o

- $2.50/M input, $10/M output Vantage
- Use: Interviewer, Orchestrator

Cost-Effective: GPT-4o Mini

- $0.15/M input, $0.60/M output Vantage
- Use: Question Generator, Code analysis

Cost Estimate: $150-250/เดือน (1,000 นักศึกษา, 5 interviews each)

4.2 Multi-Agent Framework: LangGraph

```python
from langgraph.graph import StateGraph, END

class AssessmentState(TypedDict):
    student_id: str
    questions: List[dict]
    current_q_index: int
    student_answers: List[str]
    scores: List[float]

workflow = StateGraph(AssessmentState)
workflow.add_node("present_question", interviewer_agent)
workflow.add_node("evaluate", evaluator_agent)
workflow.add_node("generate_feedback", feedback_agent)
```

4.3 Vector Database: Weaviate

Why:

- Hybrid search (semantic + keyword)
- GraphQL API
- OpenAI integration

Setup:

```python
import weaviate

client = weaviate.connect_to_weaviate_cloud(
    cluster_url="...",
    auth_credentials=Auth.api_key("...")
)

# Create schema
client.collections.create(
    name="CourseMaterial",
    vectorizer_config=Configure.Vectorizer.text2vec_openai()
)
```

4.4 Backend: FastAPI

Why:

- Async support
- Auto OpenAPI docs
- WebSocket support

Structure:

```
backend/
├── app/
│   ├── api/routes/
│   ├── agents/
│   ├── core/
│   ├── models/
│   └── main.py
```

WebSocket Example:

```python
@app.websocket("/ws/interview/{session_id}")
async def interview_websocket(websocket: WebSocket, session_id: str):
    await websocket.accept()

    state = initialize_interview_state(session_id)

    async for event in app.astream(state):
        if event["type"] == "question":
            await websocket.send_json({"question": event["data"]})
        elif event["type"] == "waiting_answer":
            answer = await websocket.receive_text()
```

4.5 Frontend: Next.js 14

Components:

- Monaco Editor (code editing) npm GitHub
- Real-time chat interface
- Dashboard analytics

```tsx
// CodeEditor.tsx
import Editor from '@monaco-editor/react'

export function CodeEditor({ onSubmit }) {
  const [code, setCode] = useState("")

  return (
    <Editor
      height="400px"
      language="python"
      value={code}
      onChange={setCode}
      theme="vs-dark"
    />
  )
}
```

4.6 Code Execution: Piston API

Why:

- 50+ languages support
- Docker isolation
- RESTful API

Usage:

```python
import requests

response = requests.post('https://emkc.org/api/v2/piston/execute',
    json={
        'language': 'python',
        'version': '3.10.0',
        'files': [{'content': student_code}],
        'stdin': test_input
    }
)
```

Self-hosting:

```bash
docker run -d -p 2000:2000 --privileged ghcr.io/engineer-man/piston
```

4.7 Authentication: Clerk

Why:

- Pre-built UI components Clerk LangChain
- SSO support (Google, Microsoft) DevTools Academy LangChain
- Organization management Clerk
- Free tier: 10K users Medium

Integration:

```tsx
import { ClerkProvider } from '@clerk/nextjs'

export default function RootLayout({ children }) {
  return (
    <ClerkProvider>
      <html><body>{children}</body></html>
    </ClerkProvider>
  )
}
```

4.8 Database: PostgreSQL (Supabase)

Free tier:

- 500MB database Zuplo
- Authentication included DevTools Academy
- Row-Level Security Clerk

Connection:

```python
from supabase import create_client

supabase = create_client(
    supabase_url="...",
    supabase_key="..."
)

# Query
data = supabase.table('sessions').select('*').execute()
```

4.9 Deployment: Railway

Why:

- Full-stack hosting MakerKit
- Docker support MakerKit
- PostgreSQL included MakerKit
- $5/month base MakerKit

Cost Estimate (1,000 students):

- Web service: $20/month
- PostgreSQL: $10/month
- Redis: $5/month
- **Total: $40/month**

4.10 Monitoring: LangSmith

Why:

- Trace every LLM call LakeFS +2
- Debug non-determinism LangSmith
- Cost tracking DrDroid Activewizards
- Evaluation tools LakeFS

Free tier: 5K traces/month DrDroid

Setup:

```python
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "..."
```

4.11 Complete Recommended Stack

MVP (Lowest Cost):

- LLM: GPT-4o Mini (~$20/month)
- Vector DB: Chroma (Free)
- Backend: FastAPI
- Frontend: Next.js
- Auth: Clerk Free
- Database: Supabase Free
- Deployment: Railway ($20/month)
- **Total: ~$40/month**

Production (Best Quality):

- LLM: Claude 3.5 + GPT-4o Mini (~$200/month)
- Vector DB: Pinecone ($70/month)
- Backend: FastAPI on Railway ($30/month)
- Frontend: Next.js on Vercel ($20/month)
- Auth: Clerk Pro ($25/month)
- Database: Supabase Pro ($25/month)
- Monitoring: LangSmith ($39/month)
- **Total: ~$430/month**

---

5. DEVELOPMENT METHODOLOGY

5.1 Project Timeline (12 เดือน)

เดือน 1-3: IDEATION & PLANNING (25%)

กิจกรรม:

- Requirements analysis: สัมภาษณ์อาจารย์ OOP 5-10 ท่าน Google Palo Alto Networks
- Literature review: อ่าน papers เกี่ยวกับ AI assessment
- System design: วาด architecture diagrams
- PoC (Proof of Concept): สร้าง simple chatbot ที่ถาม-ตอบคำถาม OOP 1 ข้อ
- Risk assessment: ระบุความเสี่ยง (เช่น LLM hallucination) Palo Alto Networks

Deliverables:

- Design document (20 หน้า)
- System architecture diagram
- Risk management plan
- Project charter

Key Milestone: PoC ทำงานได้และได้รับ feedback จากอาจารย์

เดือน 4-7: EXPERIMENTATION & MVP (35%)

กิจกรรม:

เดือน 4:

- ตั้งค่า development environment
- Implement RAG system: import เนื้อหา → chunk → embed → store
- สร้าง Question Generator Agent แรก

เดือน 5:

- Build MVP core features:
  - อาจารย์สร้าง session (manual questions)
  - นักศึกษา login และเข้า session
  - AI ถาม 3 คำถาม OOP พื้นฐาน
  - ประเมินคำตอบและให้ feedback
- Deploy MVP บน Railway

เดือน 6:

- Test MVP กับ 10 นักศึกษาจริง
- รวบรวม feedback: อะไรทำงาน? อะไรไม่ทำงาน?
- Iterate based on feedback

เดือน 7:

- เพิ่ม features:
  - Code editor integration
  - Rubric designer
  - Dashboard แสดงผล
- Create test dataset (100 คำถาม + expected answers)

Deliverables:

- Functional MVP
- Test dataset
- Initial evaluation results (accuracy, SUS score)
- Prompt library (documented)

Key Milestone: MVP functional by Month 5, tested with real users by Month 6

เดือน 8-10: PIPELINE BUILDING & TESTING (25%)

กิจกรรม:

เดือน 8:

- Build automated pipelines:
  - CI/CD: GitHub Actions (auto test + deploy)
  - Monitoring: LangSmith integration LangChain AWS
- Implement all 6 agents
- Add adaptive questioning logic

เดือน 9:

- Comprehensive testing:
  - Unit tests: แต่ละ agent ทำงานถูกต้อง
  - Integration tests: agents ทำงานร่วมกันได้
  - End-to-end tests: user flow ทั้งหมด
- LLM output testing:
  - Test prompts กับ 50 input variations
  - Measure consistency (10x same input)

เดือน 10:

- Inter-rater reliability study:
  - 100 คำตอบนักศึกษา
  - 2 อาจารย์ + AI ประเมิน
  - คำนวณ Cohen's Kappa Wikipedia Bureau of Justice Assistance
- User Acceptance Testing (UAT):
  - 15 อาจารย์ใช้ระบบ
  - 50 นักศึกษาทดสอบ
- Performance optimization: reduce latency, cost

Deliverables:

- Production pipelines
- Test results (unit, integration, e2e)
- IRR metrics ($\kappa > 0.75$ target) Bureau of Justice Assistance
- Performance benchmarks
- Complete documentation

Key Milestone: ระบบผ่าน all tests, reliability metrics achieved

---

เดือน 11-12: PRODUCTION & EVALUATION (15%)

กิจกรรม:

เดือน 11:

- Production deployment
- Final evaluation study:
  - 200 นักศึกษาใช้ระบบจริง
  - เปรียบเทียบกับ traditional assessment
  - Measure learning gains (pre-test, post-test)
- User satisfaction surveys (SUS)
- Cost analysis: ต้นทุนจริงต่อ assessment

เดือน 12:

- Data analysis และสร้าง visualizations
- Write final report (50+ หน้า):
  - Introduction, Literature Review
  - System Design, Implementation
  - Evaluation Results
  - Discussion, Conclusion
- Create presentation (20 slides)
- Prepare for demo

Deliverables:

- Deployed production system
- Final evaluation report
- User satisfaction metrics (SUS > 72) Usability Geek
- Comparison study results
- Presentation materials
- Video demo (5 นาที)

Key Milestone: Production deployment, final report complete

5.2 Development Best Practices

Agile Sprints (2 สัปดาห์):

```
Sprint Planning → Development → Testing → Review → Retrospective
```

Data Science PM

Sprint Structure:

- Week 1: Feature development
- Week 2: Testing + bug fixes
- Friday: Demo ให้อาจารย์ดู

Version Control:

```
main (production)
  └── develop (staging)
      ├── feature/question-generator
      ├── feature/code-executor
      └── bugfix/evaluation-timeout
```

Code Review:

- ทุก Pull Request ต้อง review ก่อน merge
- Checklist: tests pass, documentation updated, no security issues

Testing Strategy:

```python
# Unit test example
def test_evaluator_agent():
    answer = "Inheritance allows code reuse..."
    rubric = {...}
    score = evaluator_agent.evaluate(answer, rubric)
    assert 3 <= score <= 5  # Good answer

# Integration test
def test_full_interview_flow():
    session = create_session(questions=[...])
    result = run_interview(session, student_answers=[...])
    assert result.scores is not None
    assert result.feedback is not None
```

LLM Output Testing:

```python
# Consistency test
def test_evaluation_consistency():
    answer = "Polymorphism is..."
    scores = []

    for _ in range(10):  # Evaluate same answer 10 times
        score = evaluator_agent.evaluate(answer, rubric)
        scores.append(score)

    std_dev = np.std(scores)
    assert std_dev < 0.5  # Low variance = consistent
```

5.3 Handling LLM Unpredictability

1. Prompt Versioning:

```python
PROMPTS = {
    "evaluator_v1": "Evaluate this answer...",
    "evaluator_v2": "As an expert OOP professor, evaluate...",
    "evaluator_v3": "Using the rubric below, evaluate step-by-step..."
}

# Track which version performs best
prompt_performance = {
    "v1": {"accuracy": 0.75, "consistency": 0.8},
    "v2": {"accuracy": 0.82, "consistency": 0.85},  # Best!
    "v3": {"accuracy": 0.80, "consistency": 0.83}
}
```

2. Guardrails:

```python
def safe_llm_call(prompt, max_retries=3):
    for attempt in range(max_retries):
        try:
            response = llm.generate(prompt, timeout=30)

            # Validate output format
            if not is_valid_json(response):
                raise ValueError("Invalid format")

            # Check for hallucinations
            if contains_unsupported_claims(response):
                raise ValueError("Possible hallucination")

            return response

        except Exception as e:
            if attempt == max_retries - 1:
                # Fallback: use template response
                return generate_fallback_response()
            continue
```

3. Human-in-the-Loop:

```python
def evaluate_with_confidence(answer, rubric):
    evaluation = evaluator_agent.evaluate(answer, rubric)

    if evaluation.confidence < 0.7:
        # Flag for human review
        flag_for_instructor_review(answer, evaluation)
        return evaluation.with_disclaimer("Low confidence")

    return evaluation
```

---

6. SUCCESS CRITERIA & EVALUATION

6.1 Accuracy Metrics

Exact Agreement:

```
Exact Agreement = (Number of exact score matches) / (Total assessments) × 100%
```

- Target: **> 80%**
- Good: > 85%

Adjacent Agreement:

```
Adjacent Agreement = (Exact + Within 1 point) / (Total) × 100%
```

- Target: **> 92%**
- Good: > 95%

Mean Absolute Error (MAE):

```
MAE = Σ|AI_score - Human_score| / n
```

- Target: **< 0.5 points** (on 5-point scale)

Pearson Correlation:

```
r = correlation(AI_scores, Human_scores)
```

- Target: **> 0.85**
- Excellent: > 0.90

6.2 Inter-Rater Reliability

Study Design:

- 100 student submissions
- 3 raters: 2 human experts + 1 AI
- Calculate Cohen's Kappa for each pair

Cohen's Kappa:

```
κ = (Po - Pe) / (1 - Pe)

where:
Po = observed agreement
Pe = expected agreement by chance
```

Surge AI

Interpretation:

- < 0.40: Poor
- 0.40-0.59: Fair
- 0.60-0.74: Good
- **0.75-0.84: Excellent** ← Target
- ▌ 0.85: Near Perfect Statistics By Jim

Target: κ > 0.75 **ระหว่าง** AI **และ** human experts Bureau of Justice Assistance

6.3 User Satisfaction (SUS)

System Usability Scale:

10 **คำถาม**, 5-point Likert (1=Strongly Disagree, 5=Strongly Agree):

1. I would like to use this system frequently
2. I found the system unnecessarily complex (reverse)
3. I thought the system was easy to use
4. I would need support to use this system (reverse)
5. I found functions well integrated
6. I thought there was too much inconsistency (reverse)
7. Most people would learn this quickly
8. I found the system very cumbersome (reverse)
9. I felt confident using the system
10. I needed to learn a lot first (reverse) Wikipedia

Scoring:

```python
def calculate_sus(responses):
    # Odd items: score = rating - 1
    odd_sum = sum(responses[i] - 1 for i in [0,2,4,6,8])

    # Even items: score = 5 - rating
    even_sum = sum(5 - responses[i] for i in [1,3,5,7,9])

    sus_score = (odd_sum + even_sum) * 2.5
    return sus_score  # 0-100
```

MeasuringU

Interpretation:

- 0-50: Unacceptable (F)
- 50-62: Poor (D)
- 62-68: Marginal (C)
- **68-72: Acceptable (C+)** ← Minimum
- **72-78: Good (B)** ← Target
- 78-85: Excellent (A)
- 85-100: Best (A+) Usability Geek

Target: SUS > 72 (Good)

6.4 Performance Metrics

Response Time:

- Simple question: **< 3 seconds**
- Complex evaluation: **< 10 seconds**
- Full interview (10 questions): **< 30 minutes**

Measure at percentiles:

```python
p50 = np.percentile(response_times, 50)  # Median
p95 = np.percentile(response_times, 95)  # 95th percentile
p99 = np.percentile(response_times, 99)  # 99th percentile

# Targets
assert p50 < 3.0, "Median too slow"
assert p95 < 8.0, "95th percentile too slow"
```

Throughput:

- Target: **100 assessments/hour** per instance
- Measure: `assessments_completed / time_elapsed`

System Reliability:

```
Uptime % = (Total time - Downtime) / Total time × 100%
```

- Target: **99.5% uptime**

Error Rate:

```
Error Rate = Failed requests / Total requests × 100%
```

- Target: **< 1%**

Cost Per Assessment:

```
Cost = (LLM API costs + Infrastructure) / Number of assessments
```

- Target: **< $0.15** per assessment
- Acceptable: < $0.20

6.5 Educational Effectiveness

Learning Gains:

```
Normalized Gain = (Post-test - Pre-test) / (Max_score - Pre-test)
```

- < 0.3: Low gain
- 0.3-0.7: Medium gain
- **> 0.5**: Target

Study Design:

- Control group: Traditional assessment
- Experimental group: AI Mock Interview
- Measure: OOP test scores (pre & post)
- Compare using t-test

Feedback Quality (Survey):

- "The feedback helped me understand my mistakes" (1-5)
- "The feedback was specific and actionable" (1-5)
- **Target:** Average > 4.0/5.0

Feedback Turnaround Time:

- Traditional: 3-7 days
- AI system: **< 5 minutes** ← Target

6.6 Comparison with Traditional Assessment

Parallel Grading Study:

200 student submissions assessed by both:

- 2 human graders (traditional)
- AI system

Metrics to Compare:

- Agreement with human consensus
- Time required
- Cost
- Feedback quality (rated by students)
- Inter-rater reliability

Statistical Test:

```python
from scipy.stats import ttest_ind

# H0: AI grades = Human grades
# H1: AI grades ≠ Human grades

ai_scores = [4.2, 3.8, 4.5, ...]
human_scores = [4.0, 4.0, 4.3, ...]

t_stat, p_value = ttest_ind(ai_scores, human_scores)

if p_value > 0.05:
    print("No significant difference (good!)")
```

Effect Size (Cohen's d):

```
d = (Mean_AI - Mean_Human) / Pooled_SD
```

- Target: **d < 0.2** (small/negligible difference)

6.7 Success Criteria Summary

Minimum Acceptable (Pass):

- ✅ Accuracy: 75% exact agreement
- ✅ Correlation: $r > 0.80$
- ✅ Reliability: $\kappa > 0.75$
- ✅ Usability: SUS > 68
- ✅ Performance: Response < 10s
- ✅ Uptime: 99%

Target Performance (Good):

- ⭐ Accuracy: 85% exact, 95% adjacent
- ⭐ Correlation: $r > 0.90$
- ⭐ Reliability: $\kappa > 0.85$
- ⭐ Usability: SUS > 72
- ⭐ Performance: Response < 5s
- ⭐ Cost: < $0.15/assessment

Excellent Performance:

- 🏆 Accuracy: 90% exact
- 🏆 Correlation: $r > 0.92$
- 🏆 Reliability: $\kappa > 0.90$
- 🏆 Usability: SUS > 78
- 🏆 Learning gains: > 0.6

6.8 Evaluation Timeline

Month 6: Initial MVP evaluation

- 10 students, 30 assessments
- Pilot SUS score
- Preliminary accuracy

Month 9: Mid-project evaluation

- 50 students, 150 assessments
- Full metrics calculation
- Identify improvement areas

Month 11: Final evaluation

- 200 students, 1,000 assessments
- Comparison study with traditional
- Statistical analysis
- User surveys

Month 12: Analysis and reporting

- Data visualization
- Write results section
- Prepare presentation

---

7. RISK MANAGEMENT

7.1 Technical Risks

Risk	Likelihood	Impact	Mitigation
LLM Hallucination	High	High	• RAG system • Human-in-the-loop for low confidence • Validation layers
API Rate Limits	Medium	High	• Caching strategies • Queue system • Multiple API keys
Cost Overrun	Medium	Medium	• Use GPT-4o Mini for simple tasks • Prompt optimization • Budget alerts
Poor Accuracy	Medium	High	• Extensive prompt engineering • Iterative testing • Human validation dataset
System Downtime	Low	High	• Railway auto-scaling • Error handling • Fallback mechanisms

7.2 Project Management Risks

Risk	Mitigation
Scope Creep	Strict MVP definition, monthly reviews
Time Constraints	2-week sprints, regular milestones
Skills Gap	Online courses, mentor consultation
Insufficient Test Data	Partner with 3+ instructors for data

7.3 Ethical Risks

Algorithmic Bias:

- Mitigation: Test on diverse student populations
- Regular bias audits (by gender, language, background)
- Fairness metrics (demographic parity)

Privacy Concerns:

- Mitigation: FERPA compliance
- Data anonymization
- Clear privacy policy
- Opt-out options

Over-reliance on AI:

- Mitigation: Position as "teaching assistant"
- Always show rubric and reasoning
- Instructor override capability

---

8. DOCUMENTATION REQUIREMENTS

Technical Documentation:

1. **README.md**: Project overview, setup instructions
2. **API Documentation**: Swagger/OpenAPI auto-generated
3. **Architecture Document**: System design, diagrams
4. **Prompt Library**: All prompts with versions and performance
5. **Deployment Guide**: How to deploy to production

User Documentation:

1. **Instructor Manual**: How to create sessions, interpret results
2. **Student Guide**: How to use the system
3. **Troubleshooting Guide**: Common issues and solutions

Research Documentation:

1. **Literature Review** (10-15 หน้า)
2. **Experiment Logs**: All tests and results
3. **Evaluation Report**: Metrics, charts, analysis
4. **Final Report** (50+ หน้า): Complete project documentation

Code Documentation:

```python
# Good docstring example
def evaluate_answer(answer: str, rubric: dict) -> Evaluation:
    """
    Evaluate student answer using LLM-based agent.

    Args:
        answer: Student's written or code answer
        rubric: Dictionary containing evaluation criteria

    Returns:
        Evaluation object with score, feedback, confidence

    Example:
        >>> rubric = {"criteria": [...], "max_score": 5}
        >>> eval = evaluate_answer("Inheritance is...", rubric)
        >>> print(eval.score)  # 4.5
    """
```

---

9. DELIVERABLES

9.1 Software Deliverables

1. **Source Code**
    - GitHub repository
    - Well-structured, commented
    - README with setup instructions
2. **Deployed System**
    - Production URL (e.g., https://oop-interview.railway.app)
    - Admin panel for instructors
    - Student interface
3. **Database Backup**
    - Sample data (anonymized)
    - Schema documentation

9.2 Documentation Deliverables

1. **Final Report** (50-70 หน้า)
    - บทนำ
    - Literature Review
    - System Design
    - Implementation Details
    - Evaluation Results
    - Discussion & Conclusion
    - References
2. **Presentation Slides** (20-25 slides)
    - Problem statement
    - Solution overview
    - Demo screenshots
    - Evaluation results
    - Future work
3. **Video Demo** (5-10 นาที)
    - System walkthrough
    - Key features demonstration
    - Results visualization
4. **Poster** (A1 size)
    - For project exhibition
    - Visual summary

9.3 Evaluation Deliverables

1. **Test Dataset**
    - 100+ OOP questions
    - Expert-graded answers
    - Publicly available for research
2. **Evaluation Report**
    - All metrics (accuracy, SUS, IRR)
    - Statistical tests
    - Charts and visualizations
3. **User Study Results**
    - Survey responses
    - Interview transcripts
    - Feedback analysis

10. FUTURE WORK & EXTENSIONS

10.1 Short-term (หลัง 1 ปี)

1. **Multi-language Support**
   - รองรับหลาย programming languages (Java, C++, JavaScript)
   - Speech-to-text สำหรับ oral interviews
2. **Advanced Analytics**
   - Predictive analytics (predict student performance)
   - Learning path recommendations
   - Misconception detection dashboard
3. **Mobile App**
   - React Native app
   - Practice mode on mobile

10.2 Long-term (2-3 ปี)

1. **Multimodal Assessment**
   - Video recording analysis (body language, confidence)
   - Whiteboard diagram evaluation
   - UML diagram auto-grading
2. **Collaborative Assessment**
   - Pair programming interviews
   - Team project evaluation
3. **Cross-domain Expansion**
   - Data Structures & Algorithms
   - Database Design
   - Software Engineering
4. **Research Contributions**
   - Publish paper at SIGCSE or ICER
   - Open-source the framework
   - Benchmark dataset for community

---

11. CONCLUSION

โครงงาน AI-Driven Mock Interview System นี้ผสมผสานเทคโนโลยี **Multi-Agent LLM, RAG, และ Educational Assessment Theory** เพื่อสร้างระบบประเมินผลที่ทันสมัยและมีประสิทธิภาพ

จุดเด่นของโครงงาน:

1. **Innovative**: ใช้ multi-agent architecture ซึ่งเป็น cutting-edge technology
2. **Practical**: แก้ปัญหาจริงของอาจารย์ (grading workload) และนักศึกษา (instant feedback)
3. **Research-oriented**: มีการ evaluation อย่างเป็นระบบ, publishable results
4. **Scalable**: สถาปัตยกรรมรองรับการขยายสู่ production
5. **Ethical**: คำนึงถึง bias, privacy, และ human-in-the-loop

ความท้าทาย:

- LLM unpredictability → ต้องทำ extensive testing
- Cost management → ต้องเลือก models อย่างชาญฉลาด
- Evaluation complexity → ต้องทำ rigorous studies

ความเป็นไปได้:

- ✅ **Feasible ใน 1 ปี** ถ้าทำตาม timeline
- ✅ เทคโนโลยีพร้อมใช้ (LangGraph, Claude, GPT-4o)
- ✅ Cost reasonable ($40-400/month depending on scale)

Expected Outcomes:

- Functional system ที่ใช้งานได้จริง
- Accuracy > 80%, SUS > 72
- Complete documentation
- Potential conference paper

นี่เป็นโครงงาน Senior Project ที่ **challenging แต่ achievable** สำหรับนักศึกษาวิศวกรรมคอมพิวเตอร์ที่มีความสนใจใน AI และ Education Technology โดยจะได้เรียนรู้ทักษะที่สำคัญทั้ง LLMs, Multi-Agent Systems, Full-stack Development, และ Research Methodology ซึ่งเป็นทักษะที่ต้องการสูงในตลาดงาน AI/ML ปัจจุบัน

APPENDIX A: RECOMMENDED RESOURCES

A.1 Online Courses

LLMs & AI:

- "Generative AI with LLMs" (Coursera, DeepLearning.AI)
- "Prompt Engineering for Developers" (DeepLearning.AI, 1 hr)
- "Building Multi-Agent Systems" (DeepLearning.AI)

RAG:

- "Retrieval Augmented Generation (RAG)" (Coursera, 2 hrs)
- "Building and Evaluating Advanced RAG" (DeepLearning.AI)

Development:

- "FastAPI Tutorial" (Official docs)
- "Next.js 14 Course" (Vercel)
- "LangGraph Tutorial" (LangChain docs)

A.2 Books

AI & Education:

- "AI-Powered Learning" (Dan Fitzpatrick)
- "Understanding by Design" (Wiggins & McTighe)

OOP:

- "Head First Design Patterns"
- "Refactoring" (Martin Fowler)

Software Engineering:

- "Clean Code" (Robert Martin)
- "The Pragmatic Programmer"

A.3 Research Papers

AI Assessment:

- "Automated Assessment in CS Education: A Review" (ACM, 2024)
- "LLM-based Multi-Agent Systems" (Survey paper, 2024)

Educational AI:

- "GPT-4 as an Educational Assistant" (OpenAI, 2023)
- "Evaluating AI Grading Systems" (ICER, 2024)

A.4 Tools & Frameworks

Development:

- LangChain: https://python.langchain.com
- LangGraph: https://langchain-ai.github.io/langgraph/
- FastAPI: https://fastapi.tiangolo.com
- Next.js: https://nextjs.org

AI APIs:

- OpenAI: https://platform.openai.com
- Anthropic: https://docs.anthropic.com
- Weaviate: https://weaviate.io

Monitoring:

- LangSmith: https://smith.langchain.com

A.5 Community & Support

Discord/Forums:

- LangChain Discord
- OpenAI Developer Forum
- r/LocalLLaMA, r/MachineLearning (Reddit)

GitHub:

- Awesome LangChain: https://github.com/kyrolabs/awesome-langchain
- LangGraph Examples: https://github.com/langchain-ai/langgraph

---

APPENDIX B: SAMPLE RUBRIC

Example: Evaluating "Explain Polymorphism" Question

Criterion	Weight	Excellent (5)	Good (4)	Fair (3)	Poor (1-2)
Correctness	40%	Accurate definition, mentions method overriding/interfaces	Mostly correct with minor gaps	Partially correct	Incorrect or confused
Completeness	30%	Covers definition, benefits, examples	Covers 2 of 3	Covers 1 of 3	Incomplete
Examples	20%	Clear, relevant code example	Basic example	Vague example	No example
Terminology	10%	Uses proper OOP terms correctly	Mostly correct terms	Some incorrect terms	Poor terminology

Scoring Formula:

```
Total Score = (Correctness × 0.4) + (Completeness × 0.3) + (Examples × 0.2) + (Terminology × 0.1)
```

สิ้นสุดเอกสาร

เอกสารนี้ครอบคลุมทุกด้านของโครงงาน AI-Driven Mock Interview System สำหรับการประเมิน OOP Learning พร้อมแนวทางการพัฒนา เทคโนโลยีที่ใช้ และวิธีการประเมินความสำเร็จอย่างละเอียด เหมาะสำหรับนักศึกษาปริญญาตรีที่จะใช้เป็น Senior Project ระยะเวลา 1 ปี